

WhatNext Vision Motors

Comprehensive Setup & Implementation Guide

Customer ordering, dealer assignment, stock control and automation on Salesforce

1. Project Overview

WhatNext Vision Motors wants a better customer ordering experience and less manual work. The Salesforce solution does three things:

- ***Nearest dealer:*** *an order is linked to the dealer that matches the customer's address.*
- ***Stock control:*** *customers cannot order a vehicle that is out of stock.*
- ***Scheduled status update:*** *a daily batch job changes Pending orders to Confirmed once stock is available.*

Requirements and solutions

<i>Requirement</i>	<i>Solution</i>
<i>Store vehicles, stock, dealers, customers</i>	<i>SOX custom objects (Milestone 2)</i>
<i>Assign orders to nearest dealer</i>	<i>Record-triggered flow: Auto Assign Dealer</i>
<i>Block orders when out of stock</i>	<i>Apex trigger and handler class</i>
<i>Email reminders for test drives</i>	<i>Record-triggered flow with a scheduled path</i>
<i>Periodic stock check and status update</i>	<i>Batch Apex with Scheduled Apex</i>

Skills covered: data modelling, fields and relationships, Lightning App Builder, record-triggered flows, Apex triggers, Batch Apex and Scheduled Apex.

2. Milestone 1: Developer Org

1. Open <https://developer.salesforce.com/signup>.
2. Enter first and last name, email, Role = Developer, Company = college name, Country = India, postal code and a unique username in email format.
3. Click **Sign me up**.

4. *Open the verification email (it can take 5-10 minutes) and click **Verify Account**.*
 5. *Set a password and a security question, then click **Change Password**. You will land on the Setup home page.*
-

3. Milestone 2: Objects and Relationships

<i>Object</i>	<i>Purpose</i>	<i>Related to</i>
<code>Vehicle__c</code>	<i>Vehicle details</i>	<i>Dealer, Orders</i>
<code>Vehicle_Dealer__c</code>	<i>Authorized dealers</i>	<i>Orders</i>
<code>Vehicle_Customer__c</code>	<i>Customer details</i>	<i>Orders, Test Drives</i>
<code>Vehicle_Order__c</code>	<i>Vehicle purchases</i>	<i>Customer, Vehicle</i>
<code>Vehicle_Test_Drive__c</code>	<i>Test drive bookings</i>	<i>Customer, Vehicle</i>
<code>Vehicle_Service_Request__c</code>	<i>Service requests</i>	<i>Customer, Vehicle</i>

Create a custom object

1. Setup (gear icon) > Object Manager > Create > Custom Object.
 2. Enter Label (for example Vehicle), Plural Label (Vehicles), Record Name label (Vehicle Name) and Data Type Text.
 3. Tick **Allow Reports** and **Allow Search**, then **Save**.
 4. Repeat for the other five objects. The Dealer record name is **Dealer Name**.
-

4. Milestone 3: Tabs

1. Setup > Quick Find > Tabs. Under Custom Object Tabs click **New**.
 2. Select the object, choose any tab style, keep the defaults on the next two pages and click **Save**.
 3. Repeat for every object from Milestone 2.
-

5. Milestone 4: Lightning App

1. Setup > Quick Find > App Manager > **New Lightning App**.
2. App Name: **WhatNext Vision Motors**. Add a meaningful description and keep the default colour. Click **Next**.
3. Keep the defaults on App Options and Utility Items.

4. **Navigation Items:** add *Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports and Dashboards*.
Choose the custom objects.
5. **User Profiles:** add **System Administrator**, then **Save & Finish**.
-

6. Milestone 5: Fields and Relationships

Object	Fields (type)
Vehicle	Vehicle_Model__c (Picklist: Sedan, SUV, EV); Stock_Quantity__c (Number); Price__c (Currency); Dealer__c (Lookup to Dealer); Status__c (Picklist: Available, Out of Stock, Discontinued)
Vehicle Dealer	Dealer_Name__c (Text); Dealer_Location__c (Text); Dealer_Code__c (Auto Number); Phone__c (Phone); Email__c (Email)
Vehicle Customer	Customer_Name__c (Text); Email__c (Email); Phone__c (Phone); Address__c (Text); Preferred_Vehicle_Type__c (Picklist)

<i>Object</i>	<i>Fields (type)</i>
<i>Vehicle Order</i>	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Order_Date__c (Date); Status__c (Picklist: Pending, Confirmed, Delivered, Canceled)
<i>Vehicle Test Drive</i>	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Test_Drive_Date__c (Date); Status__c (Picklist: Scheduled, Completed, Canceled)
<i>Vehicle Service Request</i>	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Service_Date__c (Date); Issue_Description__c (Text); Status__c (Picklist: Requested, In Progress, Completed)

Create a field

1. Setup > Object Manager > choose the object > Fields & Relationships > **New**.
2. Select the data type (Picklist, Number, Lookup Relationship and so on) and click **Next**.
3. Enter the label. For a picklist, type one value per line. For a lookup, choose the related object.

4. Keep the defaults on the security and layout pages and click **Save**.

Tip: add a Lookup on Vehicle Order to Vehicle Dealer (Dealer__c) so an order can store its assigned dealer. Make sure Vehicle Test Drive has every field in the table, or the flows will not find them.

7. Milestone 6: Flows

Flow 1: Auto Assign Dealer

1. Setup > Flows > New Flow > Triggered Automations > **Record-Triggered Flow**.
2. **Object: Vehicle Order. Trigger: A record is created.**
Entry condition: Vehicle_Customer__c Is Null = False.
3. **Get Records**, label *Get Customer Information*: object Vehicle Customer; Id Equals the order's Vehicle_Customer__c ; first record only.
4. **Get Records**, label *Get Nearest Dealer*: object Vehicle Dealer; Dealer_Location__c Equals Get_Customer_Information.Address__c .
5. **Update Records**, label *Assign Dealer to Order*, using the record from Get Nearest Dealer (as in the project guide).
6. Save as **Auto Assign Dealer** and click **Activate**.

***Note:** the match is exact text equality, not true distance.
To store the dealer on the order itself, update the
triggering order and set Dealer__c to
{!Get_Nearest_Dealer.Id}.*

Flow 2: Test Drive Reminder

1. **New Flow > Record-Triggered Flow.** Object: **Vehicle Test Drive.** Trigger: **A record is created or updated.** Entry condition: Status__c Equals Scheduled.
2. Click **Add Scheduled Paths:** label *Reminder Before Test Drive*, time source *Test Drive Date*, offset *1 Days Before*.
3. **Get Records**, label *Get Customer Information:* object *Vehicle Customer*; *Id Equals the test drive's* Vehicle_Customer__c ; *first record only; store all fields.*
4. Create a **Text Template** resource named EmailSent with the message:

*Dear {!Get_Customer_Information.Name}, this is a
reminder that your test drive for
{!\$Record.Vehicle__c} is scheduled for tomorrow.
Thank you, WhatNext Vision Motors Team*

5. **Action > Send Email**, label *Send Test Drive Reminder:*
 - *Recipient Addresses = the customer's Email*

- **Subject** = Reminder: Your Test Drive is Tomorrow!
- **Body** = EmailSent
- **Rich-Text-Formatted Body** = True

6. Leave **Recipient ID** and **Related Record ID** empty. Save as **Test Drive Reminder** and click **Activate**.

8. Milestone 7: Apex Trigger and Batch Jobs

Open the **Developer Console** from the gear icon. Use **File > New** for each item below, paste the code and press **Ctrl + S**. The **Problems** tab must stay empty.

Step 1: Handler class -

VehicleOrderTriggerHandler

```
public class VehicleOrderTriggerHandler {

    public static void
    handleTrigger(List<Vehicle_Order__c>
    newOrders,
                Map<Id, Vehicle_Order__c>
    oldOrders, Boolean isBefore,
                Boolean isAfter, Boolean isInsert,
    Boolean isUpdate) {

        if (isBefore && (isInsert ||
```

```

isUpdate))
preventOrderIfOutOfStock(newOrders);
    if (isAfter && (isInsert ||
isUpdate))
updateStockOnOrderPlacement(newOrders);
    }

    // Block orders when the vehicle is out
of stock
    private static void
preventOrderIfOutOfStock(List<Vehicle_Order
__c> orders) {
        Set<Id> ids = new Set<Id>();
        for (Vehicle_Order__c o : orders)
if (o.Vehicle__c != null)
ids.add(o.Vehicle__c);
        if (ids.isEmpty()) return;

        Map<Id, Vehicle__c> m = new Map<Id,
Vehicle__c>(
            [SELECT Id, Stock_Quantity__c
FROM Vehicle__c WHERE Id IN :ids]);

        for (Vehicle_Order__c o : orders) {
            if (m.containsKey(o.Vehicle__c)
&& m.get(o.Vehicle__c).Stock_Quantity__c <=
0)
                o.addError('This vehicle is
out of stock. Order cannot be placed.');
```

```

        private static void
updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> ids = new Set<Id>();
    for (Vehicle_Order__c o : orders)
        if (o.Vehicle__c != null &&
o.Status__c == 'Confirmed')
ids.add(o.Vehicle__c);
    if (ids.isEmpty()) return;

    Map<Id, Vehicle__c> m = new Map<Id,
Vehicle__c>(
        [SELECT Id, Stock_Quantity__c
FROM Vehicle__c WHERE Id IN :ids]);

    List<Vehicle__c> upd = new
List<Vehicle__c>();
    for (Vehicle_Order__c o : orders) {
        if
(m.containsKey(o.Vehicle__c)) {
            Vehicle__c v =
m.get(o.Vehicle__c);
            if (v.Stock_Quantity__c >
0) { v.Stock_Quantity__c -= 1; upd.add(v);
}
        }
    }
    if (!upd.isEmpty()) update upd;
}
}

```

Step 2: Trigger - VehicleOrderTrigger *(object Vehicle_Order__c)*

```

trigger VehicleOrderTrigger on
Vehicle_Order__c
    (before insert, before update, after
insert, after update) {

VehicleOrderTriggerHandler.handleTrigger(Tr
igger.new, Trigger.oldMap,
    Trigger.isBefore, Trigger.isAfter,
Trigger.isInsert, Trigger.isUpdate);
}

```

Step 3: Batch class - VehicleOrderBatch

Finds Pending orders. When the vehicle has stock, it sets the order to Confirmed and reduces stock.

```

global class VehicleOrderBatch implements
Database.Batchable<sObject> {

    global Database.QueryLocator
start(Database.BatchableContext bc) {
    return
Database.getQueryLocator([SELECT Id,
Status__c, Vehicle__c
FROM Vehicle_Order__c WHERE
Status__c = 'Pending']);
}

    global void
execute(Database.BatchableContext bc,
List<Vehicle_Order__c> orderList) {

```

```

        Set<Id> ids = new Set<Id>();
        for (Vehicle_Order__c o :
orderList) if (o.Vehicle__c != null)
ids.add(o.Vehicle__c);
        if (ids.isEmpty()) return;

        Map<Id, Vehicle__c> m = new Map<Id,
Vehicle__c>(
            [SELECT Id, Stock_Quantity__c
FROM Vehicle__c WHERE Id IN :ids]);

        List<Vehicle_Order__c> os = new
List<Vehicle_Order__c>();
        Set<Id> changed = new Set<Id>();

        for (Vehicle_Order__c o :
orderList) {
            if
(m.containsKey(o.Vehicle__c)) {
                Vehicle__c v =
m.get(o.Vehicle__c);
                if (v.Stock_Quantity__c >
0) {
                    o.Status__c =
'Confirmed';
                    v.Stock_Quantity__c -=
1;
                    os.add(o);
changed.add(v.Id);
                }
            }
        }

        List<Vehicle__c> vs = new

```

```

List<Vehicle__c>();
    for (Id i : changed)
vs.add(m.get(i));
    if (!vs.isEmpty()) update vs;
    if (!os.isEmpty()) update os;
}

global void
finish(Database.BatchableContext bc) {
    System.debug('Vehicle order batch
job completed.');
```

Important: the trigger also reduces stock when an order becomes Confirmed, so stock can drop twice for one batch-confirmed order. If exact stock counts matter in your demo, remove the vehicle update from the batch.

Step 4: Scheduler class -

VehicleOrderBatchScheduler

```

global class VehicleOrderBatchScheduler
implements Schedulable {
    global void execute(SchedulableContext
sc) {
        Database.executeBatch(new
VehicleOrderBatch(), 50); // 50 = batch
size
    }
}
```

Step 5: Schedule the job

*Developer Console > Debug > Open Execute Anonymous Window. Paste the code and click Execute. Run it only once, because the same job name would be scheduled again. For midnight use '0 0 0 * * ?'.*

```
String cronExp = '0 0 12 * * ?'; // runs
daily at 12:00 PM
System.schedule('Daily Vehicle Order
Processing', cronExp, new
VehicleOrderBatchScheduler());
```

Verify: Setup > Quick Find > Scheduled Jobs. Daily Vehicle Order Processing should appear with type Scheduled Apex.

9. Testing Checklist

Test	Expected result
Create a vehicle with Stock Quantity 0, then an order for it	Error: "This vehicle is out of stock. Order cannot be placed."
Create an order for a customer whose address matches a dealer location	Flow runs and the dealer is assigned

<i>Test</i>	<i>Expected result</i>
<i>Create a test drive with Status = Scheduled</i>	<i>Reminder email is sent 1 day before the test drive date</i>
<i>Open Scheduled Jobs</i>	<i>Daily Vehicle Order Processing is listed</i>
<i>Open Setup > Flows</i>	<i>Both flows show Active</i>

10. Troubleshooting

- **Flow object box stays red:** click the object in the dropdown instead of typing it, or refresh and start a new flow. Check the object is Deployed.
- **"Provide at least one email recipient":** put the customer Email in Recipient Addresses, not in Recipient ID.
- **A merge value is missing from the list:** the lookup field is missing on the object. Create it and reopen the flow.
- **Apex "No such column":** the field API name differs from the code (for example `Stock_Quantity__c`). Fix the field or the code.
- **Scheduled job missing:** run the Execute Anonymous code again only if the job is not listed.

11. Expected Outcomes

- *A more efficient ordering process with fewer errors.*
- *No orders for out-of-stock vehicles.*
- *Less admin work through automation, so staff can focus on customers.*